

LUMA

E-Commerce Platform

Documentation, Handoff Guide & Future Roadmap

Generated for project handoff — covers what was built, how it works, admin access, and what's left for a full production launch (payments).

Contents

1. Project Overview
2. Tech Stack & Architecture
3. How This Site Was Built
4. Storefront Functionality
5. Admin Dashboard Functionality
6. Authentication, Security & OTP Password Reset
7. Transactional Email System
8. Database Schema Reference
9. Admin Access Credentials
10. Payment Integration — Stripe Setup Guide (For Later)
11. Known Limitations & Not Yet Built
12. Migrating the Backend to Django

1. Project Overview

LUMA is a generic, fully-functional e-commerce template built so a real product catalog can be dropped in later. It includes a complete storefront, a role-gated admin dashboard, real per-variant inventory, authentication with OTP-based password reset, a contact system, and transactional email hooks — all backed by a real Postgres database, not mock data.

Status
Core platform complete and tested end-to-end. Payment processing is the main piece intentionally left for later (see Section 10).

Repository

git@github.com:ZakariaRyadh/e-commerce-website.git (branch: main)

2. Tech Stack & Architecture

Framework	Next.js 16 (App Router, Turbopack) — full-stack: pages + API routes in one codebase
Database	PostgreSQL, hosted on Prisma Postgres (managed cloud, region: eu-central-1)
ORM	Prisma 7, with @prisma/adapters driver adapter
Auth	NextAuth v5 (Credentials provider, JWT sessions)
Styling	Tailwind CSS v4
State	Zustand (cart, wishlist, UI overlays) — persisted to localStorage
Email	Resend (transactional email API)
Icons	lucide-react

Why Next.js full-stack instead of a separate backend?

Keeps one codebase, faster to build and deploy, and is genuinely production-capable for a new store (comfortably handles tens of thousands of orders/month without architecture changes). All API routes are written as clean, RESTful endpoints on purpose — so if the project ever needs to split into a separate backend (e.g. Django, see Section 12), the migration is mechanical, not a rewrite.

3. How This Site Was Built

The project started from a visual mockup generated with Claude Design (claude.ai/design), exported as an HTML/CSS/JS handoff bundle. That mockup defined the visual language: minimal/clean layout, neutral palette with a single blue accent (#1B4FD8), DM Sans typography, rounded cards.

Build order:

- Next.js project scaffolded (TypeScript, Tailwind, App Router)
- Prisma schema designed and pushed to a managed Postgres database
- Authentication wired (signup/login, JWT sessions, role-based admin gate via middleware)

- Storefront pages ported from the mockup into real React components, wired to live data
- Cart, search, checkout, and account pages built with real state and real DB writes
- Admin dashboard built: products, categories, orders, customers, messages, analytics
- Per-variant inventory (size/color stock tracking) added, replacing a single stock number
- Contact form, email notifications, OTP password reset, and product reviews added
- Rate limiting added to signup, contact, and password-reset endpoints

Every feature was verified against the live database after building it — not just visually, but by creating real test accounts, placing real orders, and confirming stock actually decremented, emails were actually triggered, and permissions actually blocked unauthorized access.

4. Storefront Functionality

Home	Hero, featured products, shop-by-category (real photos/counts), value props, contact form
Collections	New Arrivals, Best Sellers (by real units sold), On Sale, category grid
Shop / Listing	Filters (category, price, rating), sort, search, mobile filter drawer
Product Detail	Gallery, size/color picker (disables sold-out combos), reviews, write-a-review, related products
Cart	Slide-out drawer, qty controls, promo code (try SAVE10), persisted across visits
Search	Global overlay, autocomplete, recent searches, suggested tags
Checkout	3-step (shipping/payment UI/review), real order creation, real stock decrement
Account	Order history, order tracking timeline, saved wishlist
Auth pages	Login, signup, forgot password, OTP reset password

5. Admin Dashboard Functionality

Dashboard	Real revenue/order/customer stats, 6-month revenue chart, recent orders
Products	Add/edit/delete, per-variant stock manager, image upload manager
Categories	Add/rename/delete (blocked while category still has products)
Orders	Search/filter, inline status updates (writes to DB, reflected in customer tracking)
Customers	Paginated + searchable, "has orders" filter, suspend/reactivate, promote to admin, delete
Messages	Contact form submissions inbox — mark read/resolved, reply by email, delete
Analytics	Order/revenue/customer breakdowns, monthly revenue chart

Security note

Suspending a user blocks new logins immediately, but an already-issued JWT session remains valid until it expires/refreshes (stateless sessions). For instant revocation, switch to database sessions later.

6. Authentication, Security & OTP Password Reset

Login & Roles

- Credentials auth (email + password), bcrypt-hashed passwords
- Two roles: CUSTOMER and ADMIN. Admin-only routes (/admin/*) are gated server-side via middleware — not just a hidden button, the route itself rejects non-admins (verified: customer login gets a 307 redirect to /login when visiting /admin)
- Account status: ACTIVE / SUSPENDED — suspended accounts cannot log in

OTP Password Reset Flow (implemented)

A real 6-digit one-time-password flow, not a magic link. This is the more "banking app"-style UX and was built end-to-end:

1. /forgot-password — user enters email. Server always responds success either way (prevents attackers from discovering which emails are registered)
2. If the email matches a real account, a 6-digit code is generated, bcrypt-hashed, stored with a 10-minute expiry, and emailed via Resend
3. /reset-password — user enters email + code + new password in one step. Server checks the code against all unexpired, unused OTPs for that user using bcrypt.compare
4. On match: password is updated and the OTP is marked used (single-use) inside a DB transaction

Tested live: wrong code rejected (400), correct code accepted (200), old password stopped working immediately after reset, new password worked on next login.

Rate Limiting

Applied to signup, contact form, forgot-password, and reset-password endpoints — simple in-memory fixed-window limiter keyed by IP address.

Scaling note

The in-memory rate limiter resets if the server restarts and doesn't share state across multiple server instances. Fine for a single-server deployment; for serverless/multi-instance production, replace with a shared store like Upstash Redis (drop-in API-compatible swap).

7. Transactional Email System

Email sending is wired through Resend (resend.com), with a safe fallback: if no API key is configured, emails are logged to the server console instead of crashing the request. This means every feature that depends on email already works today — turning on real delivery is a one-line env var change.

Emails currently wired:

- Order confirmation — sent to the customer right after a successful checkout
- Contact form received — auto-reply to the visitor who submitted the form
- Contact form admin alert — notifies the admin's email in addition to the dashboard inbox
- Password reset OTP — the 6-digit code described in Section 6

To activate real sending:

```
# .env
RESEND_API_KEY=re_XXXXXXXXXXXXXXXXXXXXXXX
```

Sign up free at resend.com, verify a sending domain (or use their shared test domain for early testing), generate an API key, paste it into `.env`. No code changes needed.

8. Database Schema Reference

User	email, passwordHash, firstName, lastName, role (CUSTOMER/ADMIN), status (ACTIVE/SUSPENDED)
PasswordResetOtp	userId, codeHash, expiresAt, usedAt — powers the OTP reset flow
Address	Saved shipping addresses per user
Category	name, slug — products belong to one category
Product	name, slug, description, price, compareAtPrice, badge, category
ProductVariant	productId, size, color, stock — real per-combo inventory (unique per size+color)
ProductImage	productId, url, position — supports admin image uploads
Review	rating, title, text, photoUrl — one review per user per product (unique constraint)
Wishlist	userId + productId pairs
Order	status, subtotal/shipping/tax/total, promoCode, shippingAddress
OrderItem	orderId, productId, name, price, qty, size, color — line items, decoupled from live product
ContactMessage	name, email, message, status (NEW/READ/RESOLVED)

Full schema lives in `prisma/schema.prisma`. Order placement runs inside a database transaction: it checks every line item's variant has enough stock, decrements stock, and creates the order atomically — if any item is out of stock, the whole order is rejected with no partial state.

9. Admin Access Credentials

Change before going live

These are seed/development credentials. Rotate the password (or create a new admin and demote/delete this one) before exposing the site publicly.

Admin URL	/admin (redirects to /login if not authenticated as an admin)
Admin email	admin@luma.test
Admin password	admin1234
Created by	prisma/seed.ts — re-running the seed script will not duplicate this user (upsert)

To promote a real account to admin instead: sign up normally as that user, then in /admin/customers use the "promote to admin" action on that account.

10. Payment Integration — Stripe Setup Guide (For Later)

Status: NOT implemented

Checkout currently captures a shipping address and card-form-shaped inputs, but no card is validated or charged, and no real payment provider is connected. This section is the guide for wiring up real payments when ready.

Recommended provider: Stripe

Stripe is the standard choice for Next.js stores — official SDKs, built-in test mode, handles PCI compliance for you (card details never touch your server).

Implementation plan:

1. Create a Stripe account, grab test API keys (STRIPE_SECRET_KEY, STRIPE_PUBLISHABLE_KEY)
2. npm install stripe @stripe/stripe-js @stripe/react-stripe-js
3. Replace the current checkout "Payment" step UI with Stripe Elements (CardElement) — this is what keeps raw card numbers off your server
4. Add POST /api/checkout/create-payment-intent — server creates a Stripe PaymentIntent for the cart total, returns its client_secret to the browser
5. Browser confirms the payment using stripe.confirmCardPayment(client_secret, ...) with the Elements card data
6. Add a Stripe webhook endpoint (POST /api/webhooks/stripe) listening for payment_intent.succeeded — this is the moment you actually create the Order row and decrement stock (don't trust the client-side confirmation alone; webhooks are the source of truth)
7. Move the existing order-creation + stock-decrement transaction (currently in POST /api/orders) into that webhook handler
8. Add STRIPE_WEBHOOK_SECRET and verify the webhook signature (stripe.webhooks.constructEvent) to stop forged requests
9. Test with Stripe's test card numbers (4242 4242 4242 4242, any future expiry/CVC) before going live
10. Switch test keys to live keys only after a full test-mode checkout has been verified end-to-end

Refunds / cancellations (related, also not built):

Once Stripe is wired, refunds are a stripe.refunds.create() call from an admin action, plus setting the order status to CANCELLED. Not built yet — was explicitly deferred alongside payment.

11. Known Limitations & Not Yet Built

Payment processing	Deferred — see Section 10
Order cancellation/refunds	Deferred — depends on payment integration

SEO	No meta descriptions per product, no sitemap.xml/robots.txt, no Open Graph tags
Product photography	Placeholder images (picsum.photos). Admin image upload exists — real photos can be uploaded per product right now, just none have been yet
Image storage	Admin-uploaded images are written to /public/uploads on local disk. This works for a single traditional server, but will NOT persist on serverless hosts like Vercel (filesystem is ephemeral there). Before deploying to Vercel, swap to cloud storage (S3, Cloudinary, or Vercel Blob) — same upload API shape, different storage backend
2FA for admin	Not built. The OTP flow exists for password reset only, not login 2FA

12. Migrating the Backend to Django

This section exists because the project deliberately started as a single Next.js full-stack app (see Section 2) instead of a separate Django backend, on the agreement that the migration path would be documented here if the site ever outgrows that setup or the team wants Python on the backend.

When to actually do this:

- Heavy background/async jobs are needed (batch emails, video/image processing, ML)
- Multiple frontends need to share one backend (web + native mobile app + partner API)
- Team is splitting into dedicated frontend/backend developers with different language preferences
- Scale genuinely requires independent scaling of API vs. frontend tiers

What moves directly (low risk):

Every API route under `src/app/api/` is already a clean REST-shaped endpoint with its own request/response contract. Each one maps to a Django view almost line-for-line:

<code>/api/signup, /api/auth/*</code>	Django: a custom auth app, or <code>django-allauth</code> . Re-implement OTP table + bcrypt check the same way (Django has <code>django.contrib.auth</code> password hashers built in)
<code>/api/products/*, /api/orders, /api/contact, /api/reviews</code>	Django REST Framework ViewSets/APIViews — one-to-one with the existing route logic
<code>/api/admin/*</code>	DRF views + a permission class checking <code>request.user.role == 'ADMIN'</code> , mirroring the current middleware check

Database migration (the DB section):

The Postgres database itself does not need to move — Django can point at the exact same database. The work is translating `prisma/schema.prisma` into Django models with matching table/column names (or using `inspectdb` to reverse-engineer Django models from the existing tables automatically):

```
# From inside a fresh Django project, pointed at the same DATABASE_URL:
python manage.py inspectdb > models_draft.py
# Then clean up: add db_table = '...' Meta options to match exact
# existing Prisma-generated table names, fix relation names, add
# verbose_name/related_name to match the app's existing usage.
```

Key mapping notes: Prisma's `@id @default(cuid())` String fields become Django CharField primary keys (or swap to Django's default UUID/auto-increment if starting fresh — but matching `cuid()` means zero downtime, since existing rows keep their IDs). Prisma enums (Role, OrderStatus, UserStatus, StockStatus-equivalent, ContactStatus) map directly to Django TextChoices. The ProductVariant unique constraint ([productId, size, color]) becomes a Meta.unique_together or UniqueConstraint.

What does NOT move (frontend stays):

The React/Next.js frontend keeps working unchanged as long as the new Django API returns the same JSON shapes at the same URL paths — point the existing `fetch()` calls at the new backend's base URL (or proxy through Next.js rewrites during a gradual cutover) and the UI doesn't need to know anything changed underneath it.

Suggested migration order:

- 1. Stand up Django + DRF pointed at a copy of the production DB (or the same DB in a staging window)
- 2. Recreate models via inspectdb, verify they match Prisma's schema exactly (field types, nullability)
- 3. Port one route family at a time, starting with the lowest-risk one (e.g. `/api/products`, read-only)
- 4. Run both backends side-by-side temporarily, route traffic gradually (feature flag or path-based split)
- 5. Port auth last, since session/JWT handling differs the most between NextAuth and Django
- 6. Decommission the Next.js API routes once Django handles 100% of traffic and is verified stable